Compositional Constructive Metamorphism for Testing Constraint Checking Implementations (Appendix)

Anonymous Authors

This appendix provides a self-contained soundness proof for CCM. We first formalize the evaluation semantics of constraint checking (CC), including both the truth value and the link set (Appx. I). We then define when a primitive unit (PU) is *precise*, which is the key invariant maintained throughout the methodology (Appx. II). Next, we prove three lemmas corresponding to the three steps of CCM: extraction preserves precision, composition preserves precision, and follow-up generation yields assertions that a correct CC implementation always satisfies (Appx. III). Finally, we combine these lemmas to establish the overall soundness theorem (Appx. IV).

I. Semantics of Constraint Checking

To give the proof of the theorem, we first give a comprehensive explanation of Constraint Checking (CC). For a given input (P, S), a CC implementation will return a truth value T and a link set L as the evaluation result. To ease discussions, we assume that every variable in the input constraint S is uniquely bound by one quantifier, e.g., constraints like $(\forall x \in (S_1))$ and $(\forall x \in (S_2))$ will not be considered because it is logically equivalent to $(\forall x \in (S_1))$ and $(\forall y \in (S_2[y/x]))$ through alpha-renaming. We formally define the evaluation rules for the truth value and the link set.

First, we define some concepts that will be used in our subsequent discussion of evaluation rules. We define the assignment set that contains variable bindings which have already been applied by quantifiers during the evaluation, formally: $\alpha = \{v_1 \mapsto e_1, v_2 \mapsto e_2, \dots\}$ where $v_i \mapsto e_i$ represents that variable v_i is bound to element e_i. We define the truth value evaluation as a function $\mathcal{T}[S]_\alpha$ and the link set generation as a function $\mathcal{L}[S]_\alpha$, where α can be omitted if empty. The context pool P is implicitly referenced in the evaluation of both functions, as evaluating S may involve accessing elements from the context sets defined in P. When we need to make this pool dependency explicit, we write $\mathcal{T}_P[S]_\alpha$ and $\mathcal{L}_P[S]_\alpha$.

A. The truth value evaluation

The truth value T is a Boolean value that indicates whether the constraint S is satisfied (T) or violated (F) against the context pool P. In the following, we list CC's truth value evaluation rules ($\mathcal{T}[S]_\alpha$) by the categories of the operators.

1) Quantifiers, i.e., $\forall$, $\exists$: The detailed truth evaluation rules for quantifiers in CC are defined as follows, which are similar to the evaluation of first-order-logic (FOL) formulas:

$$\mathcal{T}[\forall v \in C(S)]_\alpha = \begin{cases} \mathsf{T}, & \text{if } C = \emptyset \\ \bigwedge_{e \in C} \mathcal{T}[S]_{\alpha \cup \{v \mapsto e\}}, & \text{otherwise.} \end{cases} \quad (1)$$

$$\mathcal{T}[\exists v \in C(S)]_\alpha = \begin{cases} \mathsf{F}, & \text{if } C = \emptyset \\ \bigvee_{e \in C} \mathcal{T}[S]_{\alpha \cup \{v \mapsto e\}}, & \text{otherwise.} \end{cases} \quad (2)$$

2) Connectives, i.e., and, or, implies*:* The evaluation rules for logical connectives naturally follow standard Boolean algebra. They evaluate the truth value of compound constraints by recursively evaluating their sub-constraints:

$$\mathcal{T}[(S_1) \text{ and } (S_2)]_\alpha = \mathcal{T}[S_1]_\alpha \wedge \mathcal{T}[S_2]_\alpha. \quad (3)$$

$$\mathcal{T}[(S_1) \text{ or } (S_2)]_\alpha = \mathcal{T}[S_1]_\alpha \vee \mathcal{T}[S_2]_\alpha. \quad (4)$$

$$\mathcal{T}[(S_1) \text{ implies } (S_2)]_\alpha = \neg\mathcal{T}[S_1]_\alpha \vee \mathcal{T}[S_2]_\alpha. \quad (5)$$

3) Negation, i.e., not*:* The negation operator simply inverts the truth value of the underlying constraint, which is evaluated as:

$$\mathcal{T}[\text{not } (S)]_\alpha = \neg\mathcal{T}[S]_\alpha. \quad (6)$$

4) Bfunctions, i.e., $bf(\dots)$: The evaluation of a bfunction $bf(\dots)$ is determined by the specific semantics of the bfunction:

$$\mathcal{T}[bf(\dots)]_\alpha = \text{the evaluation result of } bf(\dots) \text{ under the variable bindings in } \alpha. \quad (7)$$

B. The link set generation

The link set L contains links that explain how a specific consistency constraint is satisfied or violated by listing the explicitly related elements. A link is a set of variable bindings: $l = \{v_1 \mapsto e_1, v_2 \mapsto e_2, \dots\}$ which means concurrently binding $v_i \mapsto e_i$ triggers the satisfaction or violation.

To illustrate the link set generation rules of CC, we first define two structural operators over link sets, which align with the semantic aggregation mentioned in Sec. III.D.

Union. The union ($\cup$) between link sets is the same as the standard mathematical set operation. It is used to express an alternative relationship (e.g., satisfying an or connective only requires satisfying any of its operands). The generalized big

union ($\bigcup$) extends this operation to a family of link sets, aggregating all variable bindings from multiple sets into a single comprehensive set.

Cartesian Product. The Cartesian product ($\otimes$) concatenates two link sets by taking the union of variable bindings for every pair of links formed by selecting one link from L_1 and one from L_2, i.e., $L_1 \otimes L_2 = \{l_1 \cup l_2 \mid l_1 \in L_1 \wedge l_2 \in L_2\}$, if $L_1 \neq \emptyset$ and $L_2 \neq \emptyset$; crucially, to prevent the loss of evaluation evidence when one branch generates no links, it falls back to $L_1 \cup L_2$ if either set is empty.

We then list the link set generation rules ($\mathcal{L}[S]_\alpha$) by categories of the operators.

1) Quantifiers, i.e., $\forall$, $\exists$: For quantifiers, the link set captures the specific element bindings that determine the overall truth value. A universal quantifier ($\forall$) is violated if any single element fails the constraint, while an existential quantifier ($\exists$) is satisfied if any single element meets it. The rules aggregate these critical bindings accordingly:

$$\mathcal{L}[\forall v \in C(S)]_\alpha = \bigcup_{e_i \in C \wedge \mathcal{T}[S]_{\alpha \cup \{v \mapsto e_i\}} = \mathsf{F}} (\{\{v \mapsto e_i\}\} \otimes \mathcal{L}[S]_{\alpha \cup \{v \mapsto e_i\}}). \quad (8)$$

$$\mathcal{L}[\exists v \in C(S)]_\alpha = \bigcup_{e_i \in C \wedge \mathcal{T}[S]_{\alpha \cup \{v \mapsto e_i\}} = \mathsf{T}} (\{\{v \mapsto e_i\}\} \otimes \mathcal{L}[S]_{\alpha \cup \{v \mapsto e_i\}}). \quad (9)$$

Note that $\forall$ does not generate links for elements that satisfy the constraint because they do not contribute to the violation, and $\exists$ does not generate links for elements that violate the constraint because they do not contribute to the satisfaction. Only the critical bindings that directly lead to the overall truth value are captured in the link set.

2) Connectives, i.e., $\mathsf{and}, \mathsf{or}, \mathsf{implies}$*:* Logical connectives build their link sets based on how their operands contribute to the truth value. When a connective requires concurrent conditions (e.g., both operands of an and being true, or both operands of an or being false), we use the Cartesian product ($\otimes$). When a connective allows alternative conditions, we use the union ($\bigcup$).

$$\mathcal{L}[(S_1) \text{ and } (S_2)]_\alpha = \begin{cases} \mathcal{L}[S_1]_\alpha \otimes \mathcal{L}[S_2]_\alpha, & \text{if } \mathcal{T}[S_1]_\alpha = \mathsf{T} \wedge \mathcal{T}[S_2]_\alpha = \mathsf{T} \\ \bigcup_{S_i \in \{S_1, S_2\} \wedge \mathcal{T}[S_i]_\alpha = \mathsf{F}} \mathcal{L}[S_i]_\alpha, & \text{otherwise} \end{cases}. \quad (10)$$

$$\mathcal{L}[(S_1) \text{ or } (S_2)]_\alpha = \begin{cases} \mathcal{L}[S_1]_\alpha \otimes \mathcal{L}[S_2]_\alpha, & \text{if } \mathcal{T}[S_1]_\alpha = \mathsf{F} \wedge \mathcal{T}[S_2]_\alpha = \mathsf{F} \\ \bigcup_{S_i \in \{S_1, S_2\} \wedge \mathcal{T}[S_i]_\alpha = \mathsf{T}} \mathcal{L}[S_i]_\alpha, & \text{otherwise} \end{cases}. \quad (11)$$

$$\mathcal{L}[(S_1) \text{ implies } (S_2)]_\alpha = \begin{cases} \mathcal{L}[S_1]_\alpha \otimes \mathcal{L}[S_2]_\alpha, & \text{if } \mathcal{T}[S_1]_\alpha = \mathsf{T} \wedge \mathcal{T}[S_2]_\alpha = \mathsf{F} \\ \mathcal{L}[S_1]_\alpha \cup \mathcal{L}[S_2]_\alpha, & \text{if } \mathcal{T}[S_1]_\alpha = \mathsf{F} \wedge \mathcal{T}[S_2]_\alpha = \mathsf{T} \\ \mathcal{L}[S_1]_\alpha, & \text{if } \mathcal{T}[S_1]_\alpha = \mathsf{F} \wedge \mathcal{T}[S_2]_\alpha = \mathsf{F} \\ \mathcal{L}[S_2]_\alpha, & \text{if } \mathcal{T}[S_1]_\alpha = \mathsf{T} \wedge \mathcal{T}[S_2]_\alpha = \mathsf{T} \end{cases}. \quad (12)$$

It is worth noting that the implication (S_1) implies (S_2) is logically equivalent to (not (S_1)) or (S_2). While its link set generation could technically be formulated using the $\bigcup$ operator similar to the preceding rules, we explicitly expand it into four distinct cases. This formulation is deliberately chosen for better readability and expressional clarity.

3) Negation, i.e., not*:* The negation operator simply inverts the truth value but preserves the exact same triggering reasons (links) of its underlying operand. Therefore, it directly inherits the link set of its sub-constraint.

Finally, we formulate a CC implementation as I_{CC}. If the implementation is correct, we have $I_{CC}(P, S) = \langle \mathcal{T}[S], \mathcal{L}[S] \rangle$.

$$\mathcal{L}[\text{not } (S)]_\alpha = \mathcal{L}[S]_\alpha. \quad (13)$$

4) Bfunctions, i.e., $bf(\dots)$: A bfunction does not generate any link because all variable bindings that contribute to its truth value are already captured by the link sets of the quantifiers that bind these variables. Therefore, we have:

$$\mathcal{L}[bf(\dots)]_\alpha = \emptyset. \quad (14)$$

II. Definition of A Precise Primitive Unit

After introducing the semantics of CC, we now move to the proof of the soundness of CCM. The key intermediate artifact of CCM is a *primitive unit* (PU), defined as a triple $u = \langle s, p, t \rangle$, where s is a (sub-)constraint formula, p is an associated context pool, and t is a *semantic tag*. The tag summarizes the semantic expectations of evaluating s under p for a correct CC implementation. Therefore, to establish soundness, we first formalize when a PU tag is *precise*.

We distinguish two kinds of PUs. A PU is *closed* if s contains no free variables and can be directly evaluated. A PU is *open* if s contains free variables and cannot be evaluated without additional bindings. Following our methodology, we restrict each open PU to contain *at most one* free variable, denoted by x.

Definition 1 (Precise Closed Units)

A closed unit $u = \langle s, p, t \rangle$ with tag $t = \langle T, L \rangle$ is *precise* if and only if, for a correct CC implementation I_{CC}, we have

$$I_{CC}(p, s) = \langle T, L \rangle.$$

Equivalently, under the formal semantics in Appx. I, $T = \mathcal{T}[s]$ and $L = \mathcal{L}[s]$.

Definition 2 (Precise Open Units)

An open unit $u = \langle s, p, t \rangle$ whose constraint s has *at most one* free variable (denoted by x), with tag $t = \langle x, C_\mathsf{T}, C_\mathsf{F} \rangle$, is *precise* if and only if C_T and C_F are disjoint sets of elements such that binding x to any element in C_T satisfies s, while binding x to any element in C_F violates s, i.e.,

$$(C_\mathsf{T} \cap C_\mathsf{F} = \emptyset) \wedge$$
$$(\forall e \in C_\mathsf{T},\ \mathcal{T}[s]_{\{x \mapsto e\}} = \mathsf{T}) \ \wedge\ (\forall e \in C_\mathsf{F},\ \mathcal{T}[s]_{\{x \mapsto e\}} = \mathsf{F}).$$

Definition 3 (Precise Primitive Units)

A unit $u = \langle s, p, t \rangle$ is *precise* if and only if it is a precise closed unit or a precise open unit.

III. Lemmas and Their Proofs

We will prove the soundness of CCM by proving the soundness of each step, stated as Lemma 1, Lemma 2, and Lemma 3 respectively.

A. Soundness of Step I

Before proving the soundness of Step I, we introduce two auxiliary functions used by CCM to infer PU tags from a source execution result. These functions are purely structural operators over link sets and will be used as building blocks in Lemma 1.

Let $\mathsf{Vars}(s)$ denote the set of variables that syntactically occur in a constraint formula s. For a link l (a set of bindings), and a variable set V, we define the *restriction* of l to V as $l\restriction_V = \{v \mapsto e \in l \mid v \in V\}$.

Definition 4 (Attribution Function Attr)

Given a link set L and a sub-constraint formula s, the attribution function projects each link to the variables relevant to s:

$$\mathsf{Attr}(L, s) = \{\, l\restriction_{\mathsf{Vars}(s)} \mid l \in L \wedge l\restriction_{\mathsf{Vars}(s)} \neq \emptyset \,\}.$$

Intuitively, Attr removes bindings irrelevant to s while preserving evidence about which element bindings contributed to the source evaluation.

Definition 5 (Extraction Function $\mathsf{Extract}$)

Given a link set L and a variable x, the extraction function collects all elements that are bound to x in any link:

$$\mathsf{Extract}(L, x) = \{\, e \mid \exists l \in L,\ (x \mapsto e) \in l \,\}.$$

In Step I, $\mathsf{Extract}$ is used when an open PU is obtained by removing a quantifier $Qx \in C(\cdot)$ and making x free: the extracted elements form the witness (or counterexample) set for x, and the remaining elements of the quantifier domain C can be conservatively assigned to the opposite context. (When $L = \emptyset$, we have $\mathsf{Extract}(L, x) = \emptyset$.)

Lemma 1 (Soundness of Step I)

Given a source test case and a correct CC evaluation result of it, the initial PUs CCM extracted from the case are always precise.

Proof. Let the source test case be $(P_{\text{src}}, S_{\text{src}})$. Since the CC implementation under test is assumed correct, its source execution result equals the formal semantics:

$$I_{CC}(P_{\text{src}}, S_{\text{src}}) = \langle T_{\text{src}}, L_{\text{src}} \rangle = \langle \mathcal{T}[S_{\text{src}}], \mathcal{L}[S_{\text{src}}] \rangle.$$

We will prove Lemma 1 by enumerating the extraction strategies CCM use in Step I.

(1) Decomposition-based extraction for connectives

Assume S_{src} is rooted at a binary connective, i.e., $S_{\text{src}} = (S_1)\ \mathsf{op}\ (S_2)$ where $\mathsf{op} \in \{\mathsf{and}, \mathsf{or}, \mathsf{implies}\}$. Step I constructs candidate closed PUs $u_i = \langle S_i, P_{\text{src}}, \langle T_i, L_i \rangle \rangle$, where $L_i = \mathsf{Attr}(L_{\text{src}}, S_i)$, and T_i is either deduced from T_{src} when unambiguous, or inferred from the non-emptiness of L_i when the branch truth is ambiguous. We prove that whenever such a PU is produced, it is a *precise closed unit*, i.e., $T_i = \mathcal{T}[S_i]$ and $L_i = \mathcal{L}[S_i]$.

We give the argument in detail for $\mathsf{op} = \mathsf{and}$; the other two connectives follow analogously from their corresponding rules in Eqs. (4)–(5) and Eqs. (11)–(12).

For $S_{\text{src}} = (S_1)\ \mathsf{and}\ (S_2)$, by Eq. (3),

$$T_{\text{src}} = \mathcal{T}[S_{\text{src}}] = \mathcal{T}[S_1] \wedge \mathcal{T}[S_2].$$

Case 1 (truth-determinable). If $T_{\text{src}} = \mathsf{T}$, then necessarily $\mathcal{T}[S_1] = \mathsf{T}$ and $\mathcal{T}[S_2] = \mathsf{T}$; hence setting $T_i = \mathsf{T}$ is correct. Moreover, by Eq. (10) (the T branch),

$$L_{\text{src}} = \mathcal{L}[S_{\text{src}}] = \mathcal{L}[S_1] \otimes \mathcal{L}[S_2].$$

Under our normal-form assumption that bound variables are uniquely named (alpha-renaming if needed), $\mathsf{Vars}(S_1)$ and $\mathsf{Vars}(S_2)$ are disjoint. Therefore, restricting any link $l_1 \cup l_2 \in \mathcal{L}[S_1] \otimes \mathcal{L}[S_2]$ to $\mathsf{Vars}(S_1)$ yields exactly l_1. By Definition 4, $L_1 = \mathsf{Attr}(L_{\text{src}}, S_1) = \mathcal{L}[S_1]$; symmetrically $L_2 = \mathcal{L}[S_2]$. Consequently, for each produced u_i, its tag satisfies $\langle T_i, L_i \rangle = \langle \mathcal{T}[S_i], \mathcal{L}[S_i] \rangle$, and thus u_i is precise by Definition 1.

Case 2 (truth-ambiguous). If $T_{\text{src}} = \mathsf{F}$, Eq. (3) only implies that at least one of $\mathcal{T}[S_1], \mathcal{T}[S_2]$ is F, but does not determine which one(s). In this case, Step I uses $L_i = \mathsf{Attr}(L_{\text{src}}, S_i)$ as evidence and only outputs branch PUs with $L_i \neq \emptyset$, assigning them $T_i = \mathsf{F}$ (the absorbing truth value for and).

To justify this inference, apply Eq. (10) (the F branch):

$$L_{\text{src}} = \mathcal{L}[S_{\text{src}}] = \bigcup_{S_j \in \{S_1, S_2\}\ \wedge\ \mathcal{T}[S_j] = \mathsf{F}} \mathcal{L}[S_j].$$

Thus, if a branch S_i is *true*, its link set $\mathcal{L}[S_i]$ is not included in the union and contributes no bindings to L_{src} over $\mathsf{Vars}(S_i)$, implying $\mathsf{Attr}(L_{\text{src}}, S_i) = \emptyset$. Conversely, if $\mathsf{Attr}(L_{\text{src}}, S_i) \neq \emptyset$, then necessarily $\mathcal{T}[S_i] = \mathsf{F}$ and (by the same projection argument as above) $\mathsf{Attr}(L_{\text{src}}, S_i) = \mathcal{L}[S_i]$. Therefore, every branch PU that Step I actually outputs in this case has tag $\langle T_i, L_i \rangle = \langle \mathsf{F}, \mathcal{L}[S_i] \rangle = \langle \mathcal{T}[S_i], \mathcal{L}[S_i] \rangle$ and is precise by Definition 1. (If $L_i = \emptyset$, Step I discards the branch PU; there is nothing to prove.)

This completes the proof for connective-rooted decomposition for and. The identical reasoning pattern strictly applies to or and $\mathsf{implies}$. Hence, every extracted connective branch PU is precise.

(2) Decomposition-based extraction for quantifiers

Assume S_{src} is rooted at a quantifier $Q \in \{\forall, \exists\}$, i.e., $S_{\text{src}} = Qx \in C\,(s)$, where C is the quantifier domain (a context set) provided by P_{src}. Step I removes the quantifier and outputs an

open PU $u = \langle s, P_{\text{src}}, \langle x, C_{\mathsf{T}}, C_{\mathsf{F}} \rangle \rangle$ with $C_{\mathsf{T}}, C_{\mathsf{F}}$ derived from L_{src} using Extract.

By the quantifier link rules, $L_{\text{src}} = \mathcal{L}[Qx \in C(s)]$ contains exactly the *critical* bindings of x: satisfying bindings for $\exists$ (Eq. (9)) and violating bindings for $\forall$ (Eq. (8)). Therefore, define

$$E = \mathsf{Extract}(L_{\text{src}}, x).$$

We distinguish subcases by the source truth value T_{src} (as in our methodology). Note that $\exists$ generates links only when satisfied (Eq. (9)), whereas $\forall$ generates links only when violated (Eq. (8)).

Case 1 ($T_{\text{src}} = \mathsf{T}$). If $L_{\text{src}} \neq \emptyset$, then necessarily $Q = \exists$ and Eq. (9) with Definition 5 yields

$$E = \{\, e \in C \mid \mathcal{T}[s]_{\{x \mapsto e\}} = \mathsf{T} \,\}.$$

Step I sets $C_{\mathsf{T}} = E$ and $C_{\mathsf{F}} = C \setminus E$. Then $C_{\mathsf{T}} \cap C_{\mathsf{F}} = \emptyset$, and by the characterization of E we have $\forall e \in C_{\mathsf{T}}$, $\mathcal{T}[s]_{\{x \mapsto e\}} = \mathsf{T}$ and $\forall e \in C_{\mathsf{F}}$, $\mathcal{T}[s]_{\{x \mapsto e\}} = \mathsf{F}$, hence Definition 2 holds. If instead $L_{\text{src}} = \emptyset$, then necessarily $Q = \forall$ and the universal constraint is satisfied; by Eq. (1) we have $\forall e \in C$, $\mathcal{T}[s]_{\{x \mapsto e\}} = \mathsf{T}$. Step I can set $C_{\mathsf{T}} = C$ and $C_{\mathsf{F}} = \emptyset$, which also satisfies Definition 2.

Case 2 ($T_{\text{src}} = \mathsf{F}$). If $L_{\text{src}} \neq \emptyset$, then necessarily $Q = \forall$ and Eq. (8) with Definition 5 yields

$$E = \{\, e \in C \mid \mathcal{T}[s]_{\{x \mapsto e\}} = \mathsf{F} \,\}.$$

Step I sets $C_{\mathsf{F}} = E$ and $C_{\mathsf{T}} = C \setminus E$. Then $C_{\mathsf{T}} \cap C_{\mathsf{F}} = \emptyset$ and Definition 2 holds by the characterization of E. If instead $L_{\text{src}} = \emptyset$, then necessarily $Q = \exists$ and the existential constraint is violated; by Eq. (2) we have $\forall e \in C$, $\mathcal{T}[s]_{\{x \mapsto e\}} = \mathsf{F}$. Step I can set $C_{\mathsf{T}} = \emptyset$ and $C_{\mathsf{F}} = C$, which again satisfies Definition 2.

This completes the proof for decomposition-based extraction for quantifiers.

(3) Preservation-based extraction

Step I may also preserve the whole source constraint as a closed PU $u = \langle S_{\text{src}}, P_{\text{src}}, \langle T_{\text{src}}, L_{\text{src}} \rangle \rangle$. Since $\langle T_{\text{src}}, L_{\text{src}} \rangle = \langle \mathcal{T}[S_{\text{src}}], \mathcal{L}[S_{\text{src}}] \rangle$ for a correct CC implementation, this PU is precise by Definition 1.

Moreover, Step I may preserve S_{src} as a degenerate open PU by introducing a fresh dummy variable m such that $m \notin \mathsf{Vars}(S_{\text{src}})$, together with a universal context C_{U} (the set of all possible elements). The constructed PU is

$$u_{\text{dummy}} = \langle S_{\text{src}}, P_{\text{src}}, \langle m, C_{\mathsf{T}}, C_{\mathsf{F}} \rangle \rangle,$$

where $C_{\mathsf{T}} = C_{\mathsf{U}}$ and $C_{\mathsf{F}} = \emptyset$ if $T_{\text{src}} = \mathsf{T}$, and $C_{\mathsf{T}} = \emptyset$ and $C_{\mathsf{F}} = C_{\mathsf{U}}$ if $T_{\text{src}} = \mathsf{F}$. Since m does not syntactically occur in S_{src}, binding m cannot affect the evaluation of S_{src}, i.e., for any $e \in C_{\mathsf{U}}$ we have $\mathcal{T}[S_{\text{src}}]_{\{m \mapsto e\}} = \mathcal{T}[S_{\text{src}}] = T_{\text{src}}$. Therefore, the above choice of $C_{\mathsf{T}}, C_{\mathsf{F}}$ satisfies Definition 2, and u_{dummy} is precise.

This completes the proof for preservation-based extraction.

Combining the above cases, every PU that Step I outputs is either a precise closed unit or a precise open unit, hence precise by Definition 3. This completes the proof for Lemma 1. □

B. Soundness of Step II

Step II expands the PU repository by (i) syntactically composing existing PUs using Boolean operators (not, and, or, implies), and (ii) deriving the new semantic tag by directly mirroring CC's semantics (Appx. I). Therefore, if the dependent PU tags are already precise, the derived tag remains precise.

Lemma 2 (Soundness of Step II)

Given precise PU(s), any new complex PU constructed via CCM's syntactic composition and tag derivation rules inherently remains precise.

Proof. Consider any single application of Step II that takes precise PU(s) from the repository and produces a new PU $u_{\text{new}} = \langle s_{\text{new}}, p_{\text{new}}, t_{\text{new}} \rangle$. Step II may first perform preprocessing (systematic renaming of variables and context names) to avoid name clashes. This renaming is capture-avoiding and applied consistently to the constraint and the tag; hence it preserves the values of $\mathcal{T}[\cdot]$ and $\mathcal{L}[\cdot]$ up to alpha-equivalence. Thus, it suffices to prove soundness for the core composition and tag-derivation rules.

We distinguish whether u_{new} is closed or open.

(1) Composing closed PUs

Assume the dependent PUs are closed and precise: $u = \langle s, p, \langle T, L \rangle \rangle$ and/or $u_i = \langle s_i, p_i, \langle T_i, L_i \rangle \rangle$. We show that the derived tag equals CC semantics for s_{new} under p_{new}, hence Definition 1 holds.

Unary not**.** Step II forms $s_{\text{new}} = \mathsf{not}(s)$ and sets $t_{\text{new}} = \langle \neg T, L \rangle$. Since u is precise, $T = \mathcal{T}[s]$ and $L = \mathcal{L}[s]$. By Eqs. (6) and (13), $\mathcal{T}[\mathsf{not}(s)] = \neg \mathcal{T}[s]$ and $\mathcal{L}[\mathsf{not}(s)] = \mathcal{L}[s]$. Therefore $t_{\text{new}} = \langle \mathcal{T}[s_{\text{new}}], \mathcal{L}[s_{\text{new}}] \rangle$, i.e., u_{new} is precise by Definition 1.

Binary and**.** Step II forms $s_{\text{new}} = (s_1)$ and (s_2), sets $p_{\text{new}} = p_1 \cup p_2$, and derives $t_{\text{new}} = \langle T', L' \rangle$ where $T' = T_1 \wedge T_2$ and $L' = L_1 \otimes L_2$ if $T' = \mathsf{T}$, otherwise $L' = \bigcup \{ L_i \mid T_i = \mathsf{F} \}$. Because u_1, u_2 are precise, $T_i = \mathcal{T}[s_i]$ and $L_i = \mathcal{L}[s_i]$. By Eq. (3), we have $T' = \mathcal{T}[(s_1)$ and $(s_2)]$. Moreover, Eq. (10) defines exactly the same as how we derive L', yielding $L' = \mathcal{L}[(s_1)$ and $(s_2)]$. Hence $t_{\text{new}} = \langle T', L' \rangle = \langle \mathcal{T}[s_{\text{new}}], \mathcal{L}[s_{\text{new}}] \rangle$ and u_{new} is precise by Definition 1.

Binary or **and** implies**.** Proof of these cases follow the same pattern as and.

This completes our proof for composing closed PUs.

(2) Composing open PUs

Assume the dependent PUs are open and precise: $u_1 = \langle s_1, p_1, \langle x, C_{\mathsf{T1}}, C_{\mathsf{F1}} \rangle \rangle$ and $u_2 = \langle s_2, p_2, \langle y, C_{\mathsf{T2}}, C_{\mathsf{F2}} \rangle \rangle$, where each s_i has at most one free variable. Step II unifies the free variables by a fresh z and uses capture-avoiding substitution, yielding $s_1' = s_1[z/x]$ and $s_2' = s_2[z/y]$, so the

composed constraint has at most one free variable (z). We show that the derived $\langle z, C'_{\mathsf{T}}, C'_{\mathsf{F}} \rangle$ satisfies Definition 2.

Unary not**.** For $u = \langle s, p, \langle x, C_{\mathsf{T}}, C_{\mathsf{F}} \rangle \rangle$, Step II sets $u_{\text{new}} = \langle \mathsf{not}(s), p, \langle x, C_{\mathsf{F}}, C_{\mathsf{T}} \rangle \rangle$. Since u is precise, $C_{\mathsf{T}} \cap C_{\mathsf{F}} = \emptyset$ and $\forall e \in C_{\mathsf{T}},\ \mathcal{T}[s]_{\{x \mapsto e\}} = \mathsf{T}$ and $\forall e \in C_{\mathsf{F}},\ \mathcal{T}[s]_{\{x \mapsto e\}} = \mathsf{F}$. By Eq. (6), this immediately implies $\forall e \in C_{\mathsf{F}},\ \mathcal{T}[\mathsf{not}(s)]_{\{x \mapsto e\}} = \mathsf{T}$ and $\forall e \in C_{\mathsf{T}},\ \mathcal{T}[\mathsf{not}(s)]_{\{x \mapsto e\}} = \mathsf{F}$, so Definition 2 holds.

Binary and**.** Step II forms $s_{\text{new}} = (s'_1)$ and (s'_2), sets $p_{\text{new}} = p_1 \cup p_2$, and derives $C'_{\mathsf{T}} = C_{\mathsf{T1}} \cap C_{\mathsf{T2}}$ and $C'_{\mathsf{F}} = C_{\mathsf{F1}} \cup C_{\mathsf{F2}}$. Disjointness holds because if $e \in C'_{\mathsf{T}}$, then $e \in C_{\mathsf{T1}}$ and $e \in C_{\mathsf{T2}}$, hence $e \notin C_{\mathsf{F1}}$ and $e \notin C_{\mathsf{F2}}$ (Definition 2 for u_1, u_2), so $e \notin C'_{\mathsf{F}}$. For truth correctness, take any $e \in C'_{\mathsf{T}}$; then $\mathcal{T}[s'_1]_{\{z \mapsto e\}} = \mathsf{T}$ and $\mathcal{T}[s'_2]_{\{z \mapsto e\}} = \mathsf{T}$ by Definition 2 for u_1, u_2. By Eq. (3), $\mathcal{T}[s_{\text{new}}]_{\{z \mapsto e\}} = \mathsf{T}$. Similarly, for any $e \in C'_{\mathsf{F}}$, at least one of the two sub-constraints is false under $\{z \mapsto e\}$, so Eq. (3) yields $\mathcal{T}[s_{\text{new}}]_{\{z \mapsto e\}} = \mathsf{F}$. Therefore, Definition 2 holds for the new open PU.

Binary or **and** implies**.** Proof of these cases follow the same pattern as and.

This completes our proof for composing open PUs.

Combining the above, every Step II rule application maps precise inputs to a precise output (Definitions 1–3) and proof of Lemma 2 is completed. □

C. Soundness of Step III

Step III takes a selected precise PU from the repository and constructs a follow-up test case (P_f, S_f) and an assertion A. For a closed PU, Step III may apply a randomness injection to enlarge certain context sets in P_f to increase diversity. For an open PU, Step III closes it by binding a quantifier and a target context using a semantic forcing strategy so that the expected outcome is guaranteed. We now show that, under a correct CC implementation, the follow-up execution always satisfies the derived assertion.

Before stating Lemma 3, we introduce two auxiliary definitions about link sets that will be used in the assertions and the proof.

Definition 6 (Variables in Links)

Recall that $\mathsf{Vars}(\cdot)$ denotes the set of variables in a syntactic or semantic structure. For a link l (a set of bindings), define $\mathsf{Vars}(l) = \{\, v \mid \exists e,\ (v \mapsto e) \in l \,\}$. For a link set L, define $\mathsf{Vars}(L) = \bigcup_{l \in L} \mathsf{Vars}(l)$.

Definition 7 (Link-set Extension)

For two link sets L' and L, we define the extension relation as

$$L' \ \sqsupseteq\ L \ \overset{\text{def}}{\iff}\ \forall l \in L,\ \exists l' \in L' \text{ s.t. } l \subseteq l'.$$

Intuitively, $L' \sqsupseteq L$ means every original critical binding pattern is preserved (possibly with extra bindings alongside it) after pool enlargement.

Lemma 3 (Soundness of Step III)

Given a precise PU u selected by Step III, let (P_f, S_f) and assertion A be the follow-up test case constructed from u. For a correct CC implementation, if $I_{CC}(P_f, S_f) = \langle T_f, L_f \rangle$, then A holds on $\langle T_f, L_f \rangle$.

Proof. We consider two categories depending on whether u is closed or open. We use the auxiliary notations in Definitions 6–7 throughout this proof.

(1) The selected PU is closed

Let the selected precise closed PU be $u = \langle s, p, \langle T, L \rangle \rangle$. By Definition 1, for a correct CC implementation we have $I_{CC}(p, s) = \langle T, L \rangle$, i.e., $T = \mathcal{T}[s]$ and $L = \mathcal{L}[s]$ under pool p. Step III constructs the follow-up constraint as $S_f = s$, initializes the follow-up pool as $P_f = p$ and applies randomness injection to P_f. After that, CCM gives the assertion that $T_f = T \wedge L_f \sqsupseteq L$. The derived assertion for this case requires the follow-up truth to match, and allows the follow-up link set to be an extension of the original one.

If no randomness injection is applied, then $I_{CC}(P_f, S_f) = I_{CC}(p, s) = \langle T_f, L_f \rangle$. Since the PU is precise, we have $T_f = T$ and $L_f = L$ by Definition 1. Since equality implies extension, $L_f = L$ satisfies $L_f \sqsupseteq L$ and the assertion is satisfied.

Now consider the case where Step III applies randomness injection to obtain an enlarged pool P_f^+ from P_f. Following our methodology, the injection only adds random elements to the contexts corresponding to variables in $\mathsf{Vars}(L)$, and keeps all other contexts unchanged. Let $I_{CC}(P_f^+, S_f) = \langle T^+, L^+ \rangle$. We show $T^+ = T$ and $L^+ \sqsupseteq L$, which implies the assertion holds. We prove the following structural claim by induction on the syntax of S.

To make the implicit dependency on the pool explicit, we write $\mathcal{T}_P[S]_\alpha$ and $\mathcal{L}_P[S]_\alpha$ for evaluating S under pool P with an (already-applied) assignment set α.

Claim. For any assignment set α, let P^+ be obtained from P by enlarging only the context sets of variables in $\mathsf{Vars}(\mathcal{L}_P[S]_\alpha)$ (computed under P). Then the truth value is unchanged and the new link set extends the old one, i.e., $\mathcal{T}_{P^+}[S]_\alpha = \mathcal{T}_P[S]_\alpha$ and $\mathcal{L}_{P^+}[S]_\alpha \ \sqsupseteq\ \mathcal{L}_P[S]_\alpha$.

Base cases

If S is a bfunction, then by Eq. (14) we have $\mathcal{L}_P[S]_\alpha = \emptyset$ for all P, α. Hence $\mathsf{Vars}(\mathcal{L}_P[S]_\alpha) = \emptyset$ and no injection happens; the claim is trivial.

Inductive steps

Unary not**.** If $S = \mathsf{not}(S_0)$, then by Eqs. (6) and (13) we have $\mathcal{T}_P[\mathsf{not}(S_0)]_\alpha = \neg \mathcal{T}_P[S_0]_\alpha$ and $\mathcal{L}_P[\mathsf{not}(S_0)]_\alpha = \mathcal{L}_P[S_0]_\alpha$. Therefore $\mathsf{Vars}(\mathcal{L}_P[\mathsf{not}(S_0)]_\alpha) = \mathsf{Vars}(\mathcal{L}_P[S_0]_\alpha)$ and the induction hypothesis on S_0 yields $\mathcal{T}_{P^+}[S]_\alpha = \mathcal{T}_P[S]_\alpha$ and $\mathcal{L}_{P^+}[S]_\alpha \sqsupseteq \mathcal{L}_P[S]_\alpha$.

Binary and**,** or**,** implies**.** If S is composed by a connective (and, or, or implies), then the truth of each operand is preserved by the induction hypothesis. Let $S = (S_1)\ \mathsf{op}\ (S_2)$

where $\mathsf{op} \in \{\mathsf{and}, \mathsf{or}, \mathsf{implies}\}$. By the induction hypothesis applied to S_1 and S_2, we have $\mathcal{T}_{P^+}[S_i]_\alpha = \mathcal{T}_P[S_i]_\alpha$ for $i \in \{1, 2\}$. Thus $\mathcal{T}_{P^+}[S]_\alpha = \mathcal{T}_P[S]_\alpha$.

Also, the Boolean condition that selects the semantic branch of the link-generation rule is unchanged. Concretely, in Eqs. (10)–(12), the predicates such as $\mathcal{T}[S_1]_\alpha = \mathsf{T}$ and $\mathcal{T}[S_2]_\alpha = \mathsf{F}$ hold under P if and only if they hold under P^+. Therefore, the same constructor is used to compute links before and after injection (i.e., a Cartesian product $\otimes$ in the concurrent case, or a union/selection of operand link sets in the alternative case).

It remains to show that these constructors preserve the extension relation, which is straightforward:

If $A' \sqsupseteq A$ and $B' \sqsupseteq B$, then $A' \cup B' \sqsupseteq A \cup B$. Indeed, take any $l \in A \cup B$. If $l \in A$, pick $l' \in A'$ with $l \subseteq l'$; then $l' \in A' \cup B'$. The case $l \in B$ is symmetric.

If $A' \sqsupseteq A$ and $B' \sqsupseteq B$, then $A' \otimes B' \sqsupseteq A \otimes B$. If $A = \emptyset$ or $B = \emptyset$, then by the definition of $\otimes$ we have $A \otimes B = A \cup B$ and $A' \otimes B' = A' \cup B'$, and the result follows from the union argument above. Otherwise, take any $l \in A \otimes B$; then $l = l_1 \cup l_2$ for some $l_1 \in A$ and $l_2 \in B$. Choose $l'_1 \in A'$ with $l_1 \subseteq l'_1$ and $l'_2 \in B'$ with $l_2 \subseteq l'_2$. Then $l' = l'_1 \cup l'_2 \in A' \otimes B'$ and $l \subseteq l'$.

Combining the unchanged truth-branch selection with the above two preservation facts, we have $\mathcal{L}_{P^+}[S]_\alpha \sqsupseteq \mathcal{L}_P[S]_\alpha$.

Quantifier $\forall$, $\exists$. If S is rooted at a quantifier $S = Qx \in C(S_0)$ with $Q \in \{\forall, \exists\}$, we distinguish two cases.

If $\mathcal{L}_P[S]_\alpha = \emptyset$, then $\mathsf{Vars}(\mathcal{L}_P[S]_\alpha) = \emptyset$ and thus $P^+ = P$; both truth and links are unchanged.

Assume $\mathcal{L}_P[S]_\alpha \neq \emptyset$. We treat $Q = \forall$ and $Q = \exists$.

Case $Q = \forall$. By Eq. (8), $\mathcal{L}_P[S]_\alpha \neq \emptyset$ implies that there exists some $e \in C$ such that $\mathcal{T}_P[S_0]_{\alpha \cup \{x \mapsto e\}} = \mathsf{F}$. Since P^+ only enlarges contexts and never removes elements, the original counterexample e remains in the enlarged domain of x; therefore, by Eq. (1), the universal quantifier remains violated and $\mathcal{T}_{P^+}[S]_\alpha = \mathsf{F} = \mathcal{T}_P[S]_\alpha$.

For link extension, take any $l \in \mathcal{L}_P[S]_\alpha$. By Eq. (8), there exists some counterexample $e \in C$ such that $l \in \{\{x \mapsto e\}\} \otimes \mathcal{L}_P[S_0]_{\alpha \cup \{x \mapsto e\}}$. By the induction hypothesis on S_0 under the assignment $\alpha \cup \{x \mapsto e\}$, we have $\mathcal{L}_{P^+}[S_0]_{\alpha \cup \{x \mapsto e\}} \sqsupseteq \mathcal{L}_P[S_0]_{\alpha \cup \{x \mapsto e\}}$. Using the previously established monotonicity of $\otimes$ w.r.t. $\sqsupseteq$, we obtain $\{\{x \mapsto e\}\} \otimes \mathcal{L}_{P^+}[S_0]_{\alpha \cup \{x \mapsto e\}} \sqsupseteq \{\{x \mapsto e\}\} \otimes \mathcal{L}_P[S_0]_{\alpha \cup \{x \mapsto e\}}$. Hence there exists $l' \in \mathcal{L}_{P^+}[S]_\alpha$ such that $l \subseteq l'$, i.e., $\mathcal{L}_{P^+}[S]_\alpha \sqsupseteq \mathcal{L}_P[S]_\alpha$.

Case $Q = \exists$. By Eq. (9), $\mathcal{L}_P[S]_\alpha \neq \emptyset$ implies that there exists some witness $e \in C$ such that $\mathcal{T}_P[S_0]_{\alpha \cup \{x \mapsto e\}} = \mathsf{T}$. Because the witness element remains in the enlarged domain, Eq. (2) yields $\mathcal{T}_{P^+}[S]_\alpha = \mathsf{T} = \mathcal{T}_P[S]_\alpha$. For links, the same argument as the universal case applies: any original link arises from some witness e via $\{\{x \mapsto e\}\} \otimes \mathcal{L}_P[S_0]_{\alpha \cup \{x \mapsto e\}}$, and the induction hypothesis on S_0 together with the monotonicity of $\otimes$ implies link-set extension.

This completes the induction and proves the claim, so $T^+ = T$ and $L^+ \sqsupseteq L$.

Therefore, the derived closed-PU assertion $\big(T_f = T \ \wedge \ L_f \sqsupseteq L\big)$ holds.

(2) The selected PU is open

Let the selected precise open PU be $u = \langle s_o, p_o, \langle m, C_\mathsf{T}, C_\mathsf{F} \rangle \rangle$, where s_o has at most one free variable m. By Definition 2, $C_\mathsf{T} \cap C_\mathsf{F} = \emptyset$, and $\forall e \in C_\mathsf{T},\ \mathcal{T}[s_o]_{\{m \mapsto e\}} = \mathsf{T}$ and $\forall e \in C_\mathsf{F},\ \mathcal{T}[s_o]_{\{m \mapsto e\}} = \mathsf{F}$.

Step III closes u by constructing a follow-up constraint $S_f = Qm \in C_{\mathsf{target}}(s_o)$ and a follow-up pool $P_f = p_o \cup \{C_{\mathsf{target}}\}$, where $Q \in \{\forall, \exists\}$ and C_{target} is chosen by semantic forcing. We show the follow-up execution satisfies the assertion in all forcing subcases.

Goal: forcing satisfaction ($T_f = \mathsf{T}$)

If Step III chooses $Q = \forall$, it enforces $C_{\mathsf{target}} \subseteq C_\mathsf{T}$ and applies no randomness injection. Then by Eq. (1), $\mathcal{T}[S_f] = \mathsf{T}$. Moreover, since $\forall$ generates links only when violated (Eq. (8)), we have $\mathcal{L}[S_f] = \emptyset$. Thus the assertion $\big(T_f = \mathsf{T} \ \wedge \ L_f = \emptyset\big)$ holds.

If Step III chooses $Q = \exists$, it enforces $C_{\mathsf{target}} \cap C_\mathsf{T} \neq \emptyset$ (and may inject random elements into C_{target} as noise). Pick any $e \in C_{\mathsf{target}} \cap C_\mathsf{T}$; then $\mathcal{T}[s_o]_{\{m \mapsto e\}} = \mathsf{T}$, so Eq. (2) yields $\mathcal{T}[S_f] = \mathsf{T}$. By Eq. (9), for every satisfying element in the domain, the link set contains a link that binds m to that element; hence $\mathsf{Extract}(L_f, m) \supseteq (C_{\mathsf{target}} \cap C_\mathsf{T})$. Therefore the assertion $\big(T_f = \mathsf{T} \ \wedge \ \mathsf{Extract}(L_f, m) \supseteq (C_{\mathsf{target}} \cap C_\mathsf{T})\big)$ holds.

Goal: forcing violation ($T_f = \mathsf{F}$)

If Step III chooses $Q = \forall$, it enforces $C_{\mathsf{target}} \cap C_\mathsf{F} \neq \emptyset$ (and may inject random elements into C_{target} as noise). Pick any $e \in C_{\mathsf{target}} \cap C_\mathsf{F}$; then $\mathcal{T}[s_o]_{\{m \mapsto e\}} = \mathsf{F}$, so Eq. (1) yields $\mathcal{T}[S_f] = \mathsf{F}$. Moreover, by Eq. (8), $\forall$ generates links for violating elements, so $\mathsf{Extract}(L_f, m) \supseteq (C_{\mathsf{target}} \cap C_\mathsf{F})$. Therefore the assertion $\big(T_f = \mathsf{F} \ \wedge \ \mathsf{Extract}(L_f, m) \supseteq (C_{\mathsf{target}} \cap C_\mathsf{F})\big)$ holds.

If Step III chooses $Q = \exists$, it enforces $C_{\mathsf{target}} \subseteq C_\mathsf{F}$ and applies no randomness injection. Then for all $e \in C_{\mathsf{target}}$ we have $\mathcal{T}[s_o]_{\{m \mapsto e\}} = \mathsf{F}$, so Eq. (2) yields $\mathcal{T}[S_f] = \mathsf{F}$. Also, since $\exists$ generates links only when satisfied (Eq. (9)), we have $\mathcal{L}[S_f] = \emptyset$. Thus the assertion $\big(T_f = \mathsf{F} \ \wedge \ L_f = \emptyset\big)$ holds.

In all cases, the follow-up execution result of a correct CC implementation satisfies the derived assertion. The proof of Lemma 3 is completed. □

IV. Proof of the Soundness Theorem

Theorem (Soundness of CCM)

Any correct CC implementation satisfies all test cases and assertions generated by CCM.

Proof. Consider an arbitrary run of CCM over any source test case $(P_{\mathrm{src}}, S_{\mathrm{src}})$. Assume the CC implementation under test is correct, i.e., for any input (P, S) it returns $I_{CC}(P, S) = \langle \mathcal{T}[S], \mathcal{L}[S] \rangle$ under the semantics in Appx. I.

In Step I, CCM executes the source test case and extracts a set of initial PUs from the source execution result. By Lemma 1, every PU that Step I outputs is precise (Definition 3).

In Step II, CCM repeatedly composes existing PUs and derives tags to expand the PU repository. By Lemma 2, each single rule application maps precise input PU(s) to a precise output PU. Therefore, by induction on the number of Step II compositions performed in the run, every PU in the repository after Step II is precise.

In Step III, CCM selects a PU from the repository and constructs a follow-up test case (P_f, S_f) and an assertion A. Since the selected PU is precise, Lemma 3 guarantees that the correct follow-up execution result $I_{CC}(P_f, S_f) = \langle T_f, L_f \rangle$ satisfies the derived assertion A.

Because the above argument holds for every source test case processed by CCM and for every follow-up test case and assertion it generates, any correct CC implementation satisfies all test cases and assertions generated by CCM. $\square$